

CSE46901: Introduction to Robotics

Physical Milestone 4

Professor Jaeyeon Lee
TA: Minhyuk Park

Introduction

Welcome to CSE46901. We have prepared a few milestones before the final project, in which you will use a physical robot to execute a mission given by us. The following milestones will give you an idea how to interact with [ROBOTIS' Turtlebot3 waffle Pi](#) with a [Manipulator Arm](#) using the Robot Operating System.

In this final milestone, you will learn how to use YOLO, a real time object detection algorithm based on Fully Convolutional Neural Network adapted to work with ROS on Nvidia Jetson Xavier NX. YOLO's input is a camera image fed real time from a raspberry pi v2 camera, and the output would be a bounding box showing which part of the image belongs to which object with a text label. Our TA has already modified the Turtlebot to add a **Nvidia Jetson Xavier NX, which will be used as the Remote PC to run roscore** from now on (This is due to latency reasons). We will give you one more SD card to work on Jetson. Our **TA has already burned the Jetpack's image to the SD card for you**. Please test the camera to see whether YOLO can track the plastic bottle you can find in the box of accessories we lend you in the install configuration we are providing you with.

You will find the tracking accuracy of YOLO to be acceptable but find that labeling may not always be correct. The accuracy of tracking is low because we intend to use a general YOLO v3 Tiny model trained on COCO dataset. This decision was made because we do not want to burden students with having to train a custom neural network. **You will have to think about how to cope with the poor tracking performance without training the YOLO model again.**

After testing YOLO, please **write a ROS program to control the Turtlebot's wheels such that the Turtlebot always looks at the plastic red bottle if the red bottle is within the field of view of the camera**. More precisely, if the bottle is not at the center of the field of view, the Turtlebot has to turn left or turn right so that the Turtlebot looks straight at the bottle. **Your program will have to subscribe to the bounding boxes messages from the YOLO node and identify the bounding box of the bottle. Then the Turtlebot should turn left (right) if the center of the bounding box is on the left (right) of the center of the camera's field of view.** Please submit a video clip showing how your program works, as well as the source code of your program.

Submission requirement

Hardware video demonstration submission is required for milestone4. (i.e. **You may not demonstrate your code on simulation for a pass on this milestone.**)

We are not trying to differentiate students' performance with milestones. The only grade that will be given is Pass/Fail. However, we do expect you to provide us with evidence that you have followed through the milestones by posting a video of each group following through with the tasks in Part 2. Requirements of what to be included in the video will be provided below.

Once your group is done with the video demonstration, please submit it on the blackboard along with all of your **source code**. Each group will submit one video and a zip file containing your source code. Rules for robot room usage will apply. Every member will be asked to be present during receiving/handing over the turtlebot in the robot room as our soft-policy to discourage free-riding.

Once we verify that you have submitted the milestone requirement, you will be awarded Pass/Fail. You can resubmit videos to challenge your Fail status until the final project time period so that all groups can eventually pass the milestone requirement with no penalty for resubmission. (i.e. no difference between teams who get it right the first try vs teams who retries). However, do note that there is a logistical limit of each team being able to physically interact with the turtlebot.

Instruction

Part1

(This SD card preparation step for Jetson NX was already completed by TA for you. Refer to this if you want to reflash the sd card and try this by yourself. If your sd card is corrupted, it might be faster to download and reflash the sd card image we shared with you.)

The following instructions are from our own and not from online sources. Please follow carefully and ask TA for assistance should you face any problems.

- 1) Install Ubuntu 18.04 LTS (64bit, Desktop) and necessary files (Jetpack) for Nvidia Jetson NX (Jetson Xavier NX developer kit)

<https://developer.nvidia.com/embedded/jetpack>

The compressed image file is "jetson-nx-jp451-sd-card-image.zip". After expanding the zip file, the file is "sd-blob.img".

You may require an Nvidia Account to download the file.

There are several ways to burn the image to an SD card.

As in milestone 1, you can use Raspberry Pi Imager or Ubuntu's Disks utility to write the image to a SD card.

The Raspberry Pi Imager can be downloaded here:

<https://www.raspberrypi.org/software>

In Raspberry Pi Imager, please choose "Use custom" and select "sd-blob.img".

Our TA has already burned the image to the SD card for you. But now you know how to burn it yourself if the SD card is corrupted.

- 2) Insert the SD card into Jetson NX.

Hook up Jetson NX with the portable monitor and the keyboard/mouse.

Then turn on the Turtlebot.

The Jetson NX will be powered by the OpenCR and

hence you don't need to power it separately or turn it on separately.

After you boot up Jetson NX, Jetson NX will ask you to enter the system configuration.

One of the screen will ask you to "Select Nvpmode Mode".

Please select "MODE_15W_6CORE" because we want Jetson NX to run with the maximum performance.

After setting the system configuration, the Jetson NX will reboot itself.

3) Install some software on Jetson NX:

```
sudo apt -y install curl
```

Then update Ubuntu software:

```
sudo apt-get update  
sudo apt-get upgrade
```

During upgrade, a question about "Configuration file
'/etc/nvpmode/nvpmode_t194_p3668.conf'".

You can choose Y to install the package maintainer's version.

At this point, you may want to remap the shortcut keys of Copy and Paste of your terminal to speedup the following steps.

Then reboot Jetson.

4) Setup Network:

- Run ifconfig to see the IP of Jetson NX. Remember the IP as IP_OF_Jetson.
Notice that Jetson will be the remote PC from now on.

5) Install ROS 1 Melodic (full version). Notice that you cannot use ROS 1 Noetic on Ubuntu 18.04.

- Please refer to <http://wiki.ros.org/noetic/Installation/Ubuntu>

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" >  
/etc/apt/sources.list.d/ros-latest.list'
```

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key  
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

```
curl -sSL  
'http://keyserver.ubuntu.com/pks/lookup?op=get&search=0xC1CF6E31E6BADE8868B172B  
4F42ED6FBAB17C654' | sudo apt-key add -
```

```
sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator  
python3-wstool build-essential
```

```
sudo rosdep init
```

```
rosdep update
```

6) Setup camera on Jetson:

- Use a root account to install these packages:

```
sudo apt-get install ros-melodic-image-*
```

```
sudo apt-get install ros-melodic-rqt-image-view
```

- To test the built in raspberry pi v2.1 camera on Turtlebot by this command:

```
nvgstcapture-1.0
```

This will start the preview window for the camera.

You may see that the image is upside down. But it is normal.

Then please kill the process of nvgstcapture-1.0 by pkill:

```
pkill nvgst
```

7) Setup ROS IP Address by

```
vi ~/.bashrc
```

- Add following at the end of bashrc file.
- Note that for this milestone 5 we want the Remote PC to be Jetson NX!
We do not need raspberry pi or NUC as the Remote PC.

```
source /opt/ros/melodic/setup.bash
```

```
source ~/catkin_ws/devel/setup.bash
```

```
export ROS_MASTER_URI=http://IP_OF_JetsonNX:11311
```

```
export ROS_HOSTNAME=IP_OF_JetsonNX
```

- Please make sure the first line should be "source /opt/ros/melodic/setup.bash", not "source /opt/ros/noetic/setup.bash"
- After that, execute the commands in .bashrc by sourcing it, or just start a new terminal

```
source ~/.bashrc
```

7) Setup Darknet ROS (YOLO)

- See https://github.com/leggedrobotics/darknet_ros/issues/290

```
mkdir -p catkin_ws/src
cd catkin_ws
catkin_make
source ~/.bashrc
```

```
cd ~/catkin_ws/src
git clone --recursive https://github.com/kunaltyagi/darknet_ros.git
cd darknet_ros
git branch -a
git checkout remotes/origin/opencv4
git submodule update --recursive
cd ~/catkin_ws
```

- Run the following command to resolve the missing opencv4 error that may occur later.

```
sudo ln -s /usr/include/opencv4/opencv2/ /usr/include/opencv
```

- Next you have to modify CMakeLists.txt in darknet_ros

```
sudo vi ~/catkin_ws/src/darknet_ros/darknet_ros/CMakeLists.txt
```

- At Line 20 in CMakeLists.txt, you see a "set" function that includes several parameters. Delete all parameters start with "-gencode". Then add the following parameters:

```
-gencode arch=compute_72,code=sm_72
```

FYI: 72 refers to Xavier

- Search for "link_directories" in CMakeLists.txt. There should be one "link_directories" in the file. Change the parameter of "link_directories" to

```
${CUDA_TOOLKIT_ROOT_DIR}/lib64
```

- Then build everything

```
source ~/catkin_ws/devel/setup.bash
cd ~/catkin_ws
catkin_make -DCMAKE_BUILD_TYPE=Release
source ~/catkin_ws/devel/setup.bash
```

- Change "/camera/rgb/image_raw" to "/video_source/raw_2" in ros.yaml inside darknet_ros

```
vi ~/catkin_ws/src/darknet_ros/darknet_ros/config/ros.yaml
```

8) Install packages related to publishing camera information to yolo node

This part of instruction is adapted from the following website.

https://github.com/dusty-nv/ros_deep_learning#video_source-node

```
cd ~
sudo apt-get install git cmake
git clone --recursive https://github.com/dusty-nv/jetson-inference
cd jetson-inference
mkdir build
cd build
cmake ../
```

- At this point, the installation wizard will pop up. Select "Quit" and then select "Skip".

```
make -j$(nproc)
sudo make install
sudo ldconfig
```

- Install vision related nodes

```
sudo apt-get install ros-melodic-image-transport ros-melodic-vision-msgs
```

- Finally, compile the package.

```
cd ~/catkin_ws/src
git clone https://github.com/dusty-nv/ros_deep_learning
cd ..
catkin_make
source devel/setup.bash
```

9) Test the installation by seeing the camera feed.

- Run the following two commands in two different terminal

```
roscore
```

```
roslaunch ros_deep_learning video_viewer.ros1.launch input:=csi://0 output:=display://0
```

- You should see that the camera is working.

To test Yolo's installation, read part 2

Part2

Part 2 will show you how to run YOLO. You will run this along with your control program for the milestone requirement.

The following instructions are from our own and not from online sources. Please follow carefully and ask TA for assistance should you face any problems.

0) Before you proceed with testing yolo, we need to make sure your ROS configuration is correct. After making sure that remote-pc, pi on turtlebot and Jetson NX on turtlebot is connected to the same wifi network, please change the ROS_MASTER_URI of the raspberry pi and the "remote-pc", which can be the provided NUC or your own pc, so that it points to Jetson's IP address, and **run roscore from Nvidia Jetson from now on.**

-run ifconfig on 3 machines, pi, remote-pc and Jetson NX to get their individual ip addresses.

-ssh into pi and remote-pc on separate terminal windows

-open vi on each terminal windows

-change the following line

```
export ROS_MASTER_URI=http://IP_OF_JetsonNX:11311
```

-exit and source

```
source ~/.bashrc
```

-In case of Jetson, change or add following lines to the bashrc

```
source /opt/ros/melodic/setup.bash
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://IP_OF_JetsonNX:11311
export ROS_HOSTNAME=IP_OF_JetsonNX
```

1) First, we need to write a Python script that will be used as the RGB to BGR converter. The filename of the script should be "converter.py", which contains the following code:

```
-----  
#!/usr/bin/env python  
from __future__ import print_function  
  
import roslib  
import sys  
import rospy  
import cv2  
from std_msgs.msg import String  
from sensor_msgs.msg import Image  
from cv_bridge import CvBridge, CvBridgeError  
  
class image_converter:  
  
    def __init__(self):  
        self.image_pub = rospy.Publisher("/video_source/raw_2",Image)  
  
        self.bridge = CvBridge()  
        self.image_sub = rospy.Subscriber("/video_source/raw",Image,self.callback)  
  
    def callback(self,data):  
        try:  
            cv_image = self.bridge.imgmsg_to_cv2(data, "bgr8")  
        except CvBridgeError as e:  
            print(e)  
        try:  
            self.image_pub.publish(self.bridge.cv2_to_imgmsg(cv_image, "rgb8"))  
        except CvBridgeError as e:  
            print(e)  
  
def main(args):  
    ic = image_converter()  
    rospy.init_node('image_converter', anonymous=True)  
    rospy.spin()  
  
if __name__ == '__main__':  
    main(sys.argv)  
-----
```

Please copy and paste the above code using an editor such as vi.

Suppose you put the file "converter.py" in the root directory

2) To test YOLO ROS, run the following commands in different terminals

```
roscore
```

```
roslaunch ros_deep_learning video_source.ros1.launch input:=csi://0  
output:=display://0
```

```
python converter.py
```

```
roslaunch darknet_ros darknet_ros.launch
```

- When you run converter.py, you will see many error messages initially. Please ignore these error messages associated with the camera node getting ready.
- At this point, a window will pop up. Please try to put the red bottle in front of the camera to see whether yolo can track the bottle in real time.
- You will see that Yolo often misclassifies the bottle as something else like a fire hydrant. It is because the default Yolo model can classify many different types of objects. You will have to find ways to deal with these errors in the project.

5) Before you shut down the Yolo's ROS node, check out the topic you can subscribe:

```
rostopic list
```

- Among all topics, /darknet_ros/bounding_boxes is the one that you have to know in order to do the assignment.
- Try subscribing to the publisher of this topic:

```
rostopic echo /darknet_ros/bounding_boxes
```

- Please try to read the output to see what messages the publisher provides.

Demo requirement

You will have to submit a video showing that you have successfully controlled the turtlebot with **your own original program**. Your video must first briefly explain how your code works , functionally and structurally, and must show that your code has following functions.

(1) Turtlebot can track the plastic bottle with its wheels based on information provided by YOLO. You can either move the bottle in front of the turtlebot camera's field of view to show this or show that it can track a stationary bottle.

Additionally some information on the screen to show that you are using yolo would be good. One way to do this would be to connect the hdmi monitor to Jetson while running YOLO and your program.

You do not need to contain part 2 testing in your video.

You will have to submit the source code of your program as well as a video clip showing how your program works. Please submit one video and a zip file with your group name (i.e. group_name.zip) containing your source code.

Topic explanation is not required for physical milestones.